# ns-3 Code Structure

**Dr. S. Amirtharaj**
Associate Professor
Department of Computer Applications
Mepco Schlenk Engineering College
Sivakasi – 626 005.

# What is ns-3?

- ns-3 is a discrete-event network simulator for Internet systems
  - ns-3 allows researchers to study Internet protocols and large-scale systems in a controlled environment
  - ns-3 is a new simulator (not backwards-compatible with ns-2)
- ns-3 is a free, open source software project organized around research community development and maintenance
  - the target user community is networking researchers and educators

# Boilerplate

- The first line in the file is an emacs mode line.
  - `/* -*- Mode:C++; c-file-style:"gnu"; indent-tabs-mode:nil; -*- */`
- The ns-3 simulator is licensed using the GNU General Public License.
  - ```
    /*
     * This program is free software; you can redistribute it and/or modify
     * it under the terms of the GNU General Public License version 2 as
     * published by the Free Software Foundation;
     *
     * This program is distributed in the hope that it will be useful,
     * but WITHOUT ANY WARRANTY; without even the implied warranty of
     * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
     * GNU General Public License for more details.
     *
     * You should have received a copy of the GNU General Public License
     * along with this program; if not, write to the Free Software
     * Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA  02111-1307 USA
     */
    ```
    ```

# Module Includes

- The code proper starts with a number of include statements.
  - ```
    #include "ns3/core-module.h"
    #include "ns3/simulator-module.h"
    #include "ns3/node-module.h"
    #include "ns3/helper-module.h"
    ```
    - ../../build/debug/ns3
    ```

# Ns3 Namespace

- The next line in the first.cc script is a namespace declaration.
  - `using namespace ns3;`
- Logging
  - `NS_LOG_COMPONENT_DEFINE ("FirstScriptExample");`

# Main Functions

- The next lines of the script you will find are,

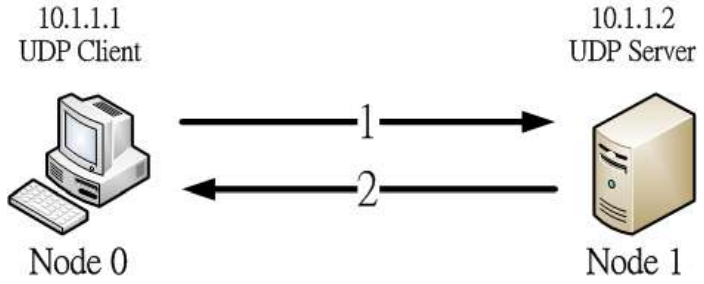
```
int
main (int argc, char *argv[])
{
```

```
LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_INFO);
LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_INFO);
```

ns3 log :
1.NS_LOG_ERROR — Log error messages;
2.NS_LOG_WARN — Log warning messages;
3.NS_LOG_DEBUG — Log relatively rare, ad-hoc debugging messages;
4.NS_LOG_INFO — Log informational messages about program progress;
5.NS_LOG_FUNCTION — Log a message describing each function called;
6.NS_LOG_LOGIC – Log messages describing logical flow within a function;
7.NS_LOG_ALL — Log everything.
8.NS_LOG_UNCOND – Log the associated message unconditionally.

# Topology Helpers

- Create the ns-3 Node objects that will represent the computers in the simulation.
  - ```
    NodeContainer nodes;
    nodes.Create (2);
    ```




- We are constructing a point to point link
  - ```
    PointToPointHelper pointToPoint;
    pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
    pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));
    ```

# Topology Helpers

- We will need to have a list of all of the NetDevice objects that are created
  - ```
    NetDeviceContainer devices;
    devices = pointToPoint.Install (nodes);
    ```
    ```

# Internet Stack Helper

- Protocol stacks installed on our nodes
  - ```
    InternetStackHelper stack;
    stack.Install (nodes);
    ```
    - TCP, UDP, IP, etc.
- Ipv4AddressHelper
  - ```
    Ipv4AddressHelper address;
    address.SetBase ("10.1.1.0", "255.255.255.0");
    ```
  - ```
    Ipv4InterfaceContainer interfaces = address.Assign (devices);
    ```
    - make the association between an IP address and a device
    ```

# Applications

- Two specializations of the core ns-3 class Application called
  - UdpEchoServerApplication
  - UdpEchoClientApplication

- UdpEchoServerHelper

```
UdpEchoServerHelper echoServer (9);

ApplicationContainer serverApps = echoServer.Install (nodes.Get (1));
serverApps.Start (Seconds (1.0));
serverApps.Stop (Seconds (10.0));
```

# Applications

- UdpEchoClientHelper

```
UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9);
echoClient.SetAttribute ("MaxPackets", UintegerValue (1));
echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.)));
echoClient.SetAttribute ("PacketSize", UintegerValue (1024));

ApplicationContainer clientApps = echoClient.Install (nodes.Get (0));
clientApps.Start (Seconds (2.0));
clientApps.Stop (Seconds (10.0));
```

# Simulator

- What we need to do at this point is to actually run the simulation.
  - `Simulator::Run`
    - scheduled events in the simulator at 1.0 seconds, 2.0 seconds and two events at 10.0 seconds
  - 

```
  Simulator::Destroy ();
  return 0;
}
```

# Building Your Script

- Drop your script into the scratch directory
  - ```
    cd ..
    cp examples/tutorial/first.cc scratch/myfirst.cc
    ```
- Now build your first example script using waf
  - ```
    ./waf
    ```
  - ```
    Waf: Entering directory '/home/craigdo/repos/ns-3-allinone/ns-3-dev/build'
    [614/708] cxx: scratch/myfirst.cc -> build/debug/scratch/myfirst_3.o
    [706/708] cxx_link: build/debug/scratch/myfirst_3.o -> build/debug/scratch/myfirst
    Waf: Leaving directory '/home/craigdo/repos/ns-3-allinone/ns-3-dev/build'
    'build' finished successfully (2.357s)
    ```
    ```

# Building Your Script

- You can now run the example
  - `./waf --run scratch/myfirst`

```
Waf: Entering directory '/home/craigdo/repos/ns-3-allinone/ns-3-dev/build'
Waf: Leaving directory '/home/craigdo/repos/ns-3-allinone/ns-3-dev/build'
'build' finished successfully (0.418s)
Sent 1024 bytes to 10.1.1.2
Received 1024 bytes from 10.1.1.1
Received 1024 bytes from 10.1.1.2
```

# Building a Wireless Network Topology

- Add the Wifi and the mobility modules

```
#include "ns3/core-module.h"
#include "ns3/simulator-module.h"
#include "ns3/node-module.h"
#include "ns3/helper-module.h"
#include "ns3/wifi-module.h"
#include "ns3/mobility-module.h"
```

- The network topology illustration follows:

```
// Default Network Topology
//
//   Wifi 10.1.3.0
//                   AP
//  *     *     *     *
//  |     |     |     |    10.1.1.0
// n5    n6    n7    n0 -------------- n1   n2   n3   n4
//                    point-to-point  |    |    |    |
//                                    ================
//                                      LAN 10.1.2.0
```

# Building a Wireless Network Topology

- For enabling or disabling logging components and for changing the number of devices created

```
bool verbose = true;
uint32_t nCsma = 3;
uint32_t nWifi = 3;

CommandLine cmd;
cmd.AddValue ("nCsma", "Number of \"extra\" CSMA nodes/devices", nCsma);
cmd.AddValue ("nWifi", "Number of wifi STA devices", nWifi);
cmd.AddValue ("verbose", "Tell echo applications to log if true", verbose);

cmd.Parse (argc,argv);

if (verbose)
  {
    LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_INFO);
    LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_INFO);
  }
```

# Building a Wireless Network Topology

- Part of the Wifi network

```
NodeContainer wifiStaNodes;
wifiStaNodes.Create (nWifi);
NodeContainer wifiApNode = p2pNodes.Get (0);
```

- Configure the PHY and channel helpers

```
YansWifiChannelHelper channel = YansWifiChannelHelper::Default ();
YansWifiPhyHelper phy = YansWifiPhyHelper::Default ();
```

  - Yet Another Network Simulator
    - M. Lacage and T. R. Henderson, “Yet Another Network Simulator,” WNS2 ns-2: The IP Network Simulator, Italy, 2006.
    - http://cutebugs.net/files/wns2-yans.pdf

# Building a Wireless Network Topology

- Create a channel object and associate it to our PHY layer object manager
  - ```
    phy.SetChannel (channel.Create ());
    ```

- NqosWifiMacHelper object to set MAC parameters
  - ```
    WifiHelper wifi = WifiHelper::Default ();
    wifi.SetRemoteStationManager ("ns3::AarfWifiManager");

    NqosWifiMacHelper mac = NqosWifiMacHelper::Default ();
    ```
  - rate control algorithm
    - AARF (Adaptive Auto-Rate Fallback ) algorithm

# Building a Wireless Network Topology

- The SSID of the infrastructure network

```
Ssid ssid = Ssid ("ns-3-ssid");
mac.SetType ("ns3::StaWifiMac",
  "Ssid", SsidValue (ssid),
  "ActiveProbing", BooleanValue (false));
```

- Create the wifi devices of these stations

```
NetDeviceContainer staDevices;
staDevices = wifi.Install (phy, mac, wifiStaNodes);
```

# Building a Wireless Network Topology

- Configure the AP (access point) node

```
mac.SetType ("ns3::ApWifiMac",
  "Ssid", SsidValue (ssid),
  "BeaconGeneration", BooleanValue (true),
  "BeaconInterval", TimeValue (Seconds (2.5)));
```

- Shares the same set of PHY-level Attributes (and channel) as the stations

```
NetDeviceContainer apDevices;
apDevices = wifi.Install (phy, mac, wifiApNode);
```

# Building a Wireless Network Topology

- Set some Attributes controlling the “position allocator” functionality

```
MobilityHelper mobility;

mobility.SetPositionAllocator ("ns3::GridPositionAllocator",
  "MinX", DoubleValue (0.0),
  "MinY", DoubleValue (0.0),
  "DeltaX", DoubleValue (5.0),
  "DeltaY", DoubleValue (10.0),
  "GridWidth", UintegerValue (3),
  "LayoutType", StringValue ("RowFirst"));
```

The number of objects layed out on a line.

# Building a Wireless Network Topology

- Need to tell them how to move
  - RandomWalk2dMobilityModel
    - random direction
    - random speed
  - ```
    mobility.SetMobilityModel ("ns3::RandomWalk2dMobilityModel",
      "Bounds", RectangleValue (Rectangle (-50, 50, -50, 50)));
    ```
    - Rectangle (double _xMin, double _xMax, double _yMin, double _yMax)
  - ```
    mobility.Install (wifiStaNodes);
    ```

# Building a Wireless Network Topology

- Want the access point to remain in a fixed position during the simulation

```
mobility.SetMobilityModel ("ns3::ConstantPositionMobilityModel");
mobility.Install (wifiApNode);
```

- Protocol stacks

```
InternetStackHelper stack;
stack.Install (csmaNodes);
stack.Install (wifiApNode);
stack.Install (wifiStaNodes);
```

# Building a Wireless Network Topology

- assign IP addresses to the device interfaces

```
Ipv4AddressHelper address;

address.SetBase ("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer p2pInterfaces;
p2pInterfaces = address.Assign (p2pDevices);

address.SetBase ("10.1.2.0", "255.255.255.0");
Ipv4InterfaceContainer csmaInterfaces;
csmaInterfaces = address.Assign (csmaDevices);

address.SetBase ("10.1.3.0", "255.255.255.0");
address.Assign (staDevices);
address.Assign (apDevices);
```

# Building a Wireless Network Topology

- Enable internetwork routing
  - ```
    Ipv4GlobalRoutingHelper::PopulateRoutingTables ();
    ```
  - ```
    Simulator::Stop (Seconds (10.0));
    ```
- Create just enough tracing to cover all three networks
  - ```
    pointToPoint.EnablePcapAll ("third");
    phy.EnablePcap ("third", apDevices.Get (0));
    csma.EnablePcap ("third", csmaDevices.Get (0), true);
    ```
  - third-0-0.pcap third-0-1.pcap third-1-0.pcap third-1-1.pcap
    ```

# Building a Wireless Network Topology

- Run the simulation, clean up and then exit the program
  - ```
    Simulator::Run ();
    Simulator::Destroy ();
    return 0;
    }
    ```
- Output
  - ```
    Waf: Entering directory `/home/craigdo/repos/ns-3-allinone/ns-3-dev/build'
    Waf: Leaving directory `/home/craigdo/repos/ns-3-allinone/ns-3-dev/build'
    'build' finished successfully (0.407s)
    Sent 1024 bytes to 10.1.2.4
    Received 1024 bytes from 10.1.3.3
    Received 1024 bytes from 10.1.2.4
    ```

# Read the pcap files

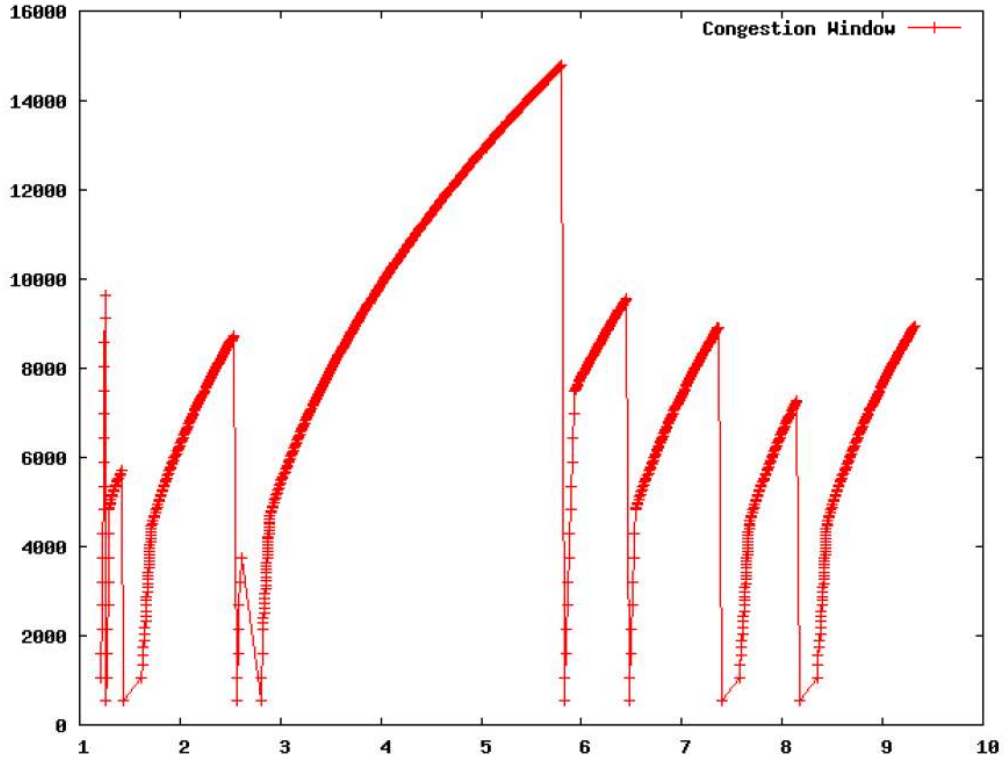
```
tcpdump -nn -tt -r third-0-1.pcap
```

```
reading from file third-0-1.pcap, link-type IEEE802_11 (802.11)
0.000025 Beacon () [6.0* 9.0 12.0 18.0 24.0 36.0 48.0 54.0 Mbit] IBSS
0.000263 Assoc Request () [6.0 9.0 12.0 18.0 24.0 36.0 48.0 54.0 Mbit]
0.000279 Acknowledgment RA:00:00:00:00:00:07
0.000357 Assoc Response AID(0) :: Succesful
0.000501 Acknowledgment RA:00:00:00:00:00:0a
0.000748 Assoc Request () [6.0 9.0 12.0 18.0 24.0 36.0 48.0 54.0 Mbit]
0.000764 Acknowledgment RA:00:00:00:00:00:08
0.000842 Assoc Response AID(0) :: Succesful
0.000986 Acknowledgment RA:00:00:00:00:00:0a
0.001242 Assoc Request () [6.0 9.0 12.0 18.0 24.0 36.0 48.0 54.0 Mbit]
0.001258 Acknowledgment RA:00:00:00:00:00:09
0.001336 Assoc Response AID(0) :: Succesful
0.001480 Acknowledgment RA:00:00:00:00:00:0a
2.000112 arp who-has 10.1.3.4 (ff:ff:ff:ff:ff:ff) tell 10.1.3.3
2.000128 Acknowledgment RA:00:00:00:00:00:09
2.000206 arp who-has 10.1.3.4 (ff:ff:ff:ff:ff:ff) tell 10.1.3.3
2.000487 arp reply 10.1.3.4 is-at 00:00:00:00:00:0a
2.000659 Acknowledgment RA:00:00:00:00:00:0a
2.002169 IP 10.1.3.3.49153 > 10.1.2.4.9: UDP, length 1024
2.002185 Acknowledgment RA:00:00:00:00:00:09
2.009771 arp who-has 10.1.3.3 (ff:ff:ff:ff:ff:ff) tell 10.1.3.4
2.010029 arp reply 10.1.3.3 is-at 00:00:00:00:00:09
2.010045 Acknowledgment RA:00:00:00:00:00:09
2.010231 IP 10.1.2.4.9 > 10.1.3.3.49153: UDP, length 1024
2.011767 Acknowledgment RA:00:00:00:00:00:0a
2.500000 Beacon () [6.0* 9.0 12.0 18.0 24.0 36.0 48.0 54.0 Mbit] IBSS
5.000000 Beacon () [6.0* 9.0 12.0 18.0 24.0 36.0 48.0 54.0 Mbit] IBSS
7.500000 Beacon () [6.0* 9.0 12.0 18.0 24.0 36.0 48.0 54.0 Mbit] IBSS
```

# Plot figures

```
./waf --run fifth > cwnd.dat 2>&1
```

```
gnuplot> set terminal png size 640,480
gnuplot> set output "cwnd.png"
gnuplot> plot "cwnd.dat" using 1:2 title 'Congestion Window' with linespoints
gnuplot> exit
```



# An Example of MANET

- Reference
  - /ns-allinone-3.10/ns-3.10/examples/wireless/wifi-hidden-terminal.cc

# Codes (1/7)

```
 7: #include "ns3/core-module.h"
 8: #include "ns3/common-module.h"
 9: #include "ns3/node-module.h"
10: #include "ns3/helper-module.h"
11: #include "ns3/mobility-module.h"
12: #include "ns3/flow-monitor-module.h"
13:
14: // ***********************************
15: //   -------------------------------
16: //   |                             |
17: //   |             * - n0          |
18: //   |                             |
19: //   |     * - n1                  |
20: //   |                             |
21: //   |                  * - n2     | 100m
22: //   |                             |
23: //   |                             |
24: //   |   * - n3                    |
25: //   |                             |
26: //   |                * - n4       |
27: //   |                             |
28: //   -------------------------------
29: //              100m
30: // ***********************************
```

# Codes (2/7)

```
40:   // 1. Create nodes
41:   NodeContainer nodes;
42:   nodes.Create (nSta);
43:
44:   // 2. Place nodes somehow, this is required by every wireless simulation
45:   MobilityHelper mobility;
46:
47:   mobility.SetPositionAllocator ("ns3::GridPositionAllocator",
48:                                  "MinX", DoubleValue (0.0),
49:                                  "MinY", DoubleValue (0.0),
50:                                  "DeltaX", DoubleValue (5.0),
51:                                  "DeltaY", DoubleValue (5.0),
52:                                  "GridWidth", UintegerValue (nSta),
53:                                  "LayoutType", StringValue ("RowFirst"));
54:
55:   mobility.SetMobilityModel ("ns3::RandomWalk2dMobilityModel",
56:                              "Bounds", RectangleValue (Rectangle (-50, 50, -50,
    50)));
57:   mobility.Install (nodes);
```

# Codes (3/7)

```
59:   // 3. Create propagation loss model
60:
61:   Ptr<MatrixPropagationLossModel> lossModel =
    CreateObject<MatrixPropagationLossModel> ();

62:   lossModel->SetDefaultLoss (0); // set loss to 0 dB
63:
64:   // 4. Create & setup wifi channel
65:   Ptr<YansWifiChannel> wifiChannel = CreateObject <YansWifiChannel> ();
66:   wifiChannel->SetPropagationLossModel (lossModel);
67:   wifiChannel->SetPropagationDelayModel (CreateObject
    <ConstantSpeedPropagationDelayModel> ());
```

# Codes (4/7)

```
69:   // 5. Install wireless devices
70:   WifiHelper wifi;
71:   wifi.SetStandard (WIFI_PHY_STANDARD_80211b);
72:   wifi.SetRemoteStationManager ("ns3::ConstantRateWifiManager",
73:                                 "DataMode",StringValue ("DsssRate2Mbps"),
74:                                 "ControlMode",StringValue ("DsssRate2Mbps"));
75:   YansWifiPhyHelper wifiPhy = YansWifiPhyHelper::Default ();
76:   wifiPhy.SetChannel (wifiChannel);
77:
78:   NqosWifiMacHelper wifiMac = NqosWifiMacHelper::Default ();
79:   NetDeviceContainer devices = wifi.Install (wifiPhy, wifiMac, nodes);
80:
81:   // 6. Install TCP/IP stack & assign IP addresses
82:   InternetStackHelper internet;
83:   internet.Install (nodes);
84:   Ipv4AddressHelper ipv4;
85:   ipv4.SetBase ("192.168.0.0", "255.255.255.0");
86:   Ipv4InterfaceContainer interface = ipv4.Assign (devices);
```

# Codes (5/7)

```
88:   // 7. Install applications: two CBR streams each saturating the channel
89:   ApplicationContainer cbrApps;
90:   uint16_t cbrPort = 1234;
91:   OnOffHelper onOffHelper ("ns3::UdpSocketFactory", InetSocketAddress
    (interface.GetAddress(9), cbrPort));
92:   onOffHelper.SetAttribute ("PacketSize", UintegerValue (1000)); //The size of
    packets sent in on state
93:   onOffHelper.SetAttribute ("OnTime",  RandomVariableValue (ConstantVariable
    (100)));
94:   onOffHelper.SetAttribute ("OffTime", RandomVariableValue (ConstantVariable
    (0)));
95:
96:   // flow :  node 0 -> node 1
97:   onOffHelper.SetAttribute ("DataRate", StringValue ("1000000bps"));
98:   onOffHelper.SetAttribute ("StartTime", TimeValue (Seconds (1.000000)));
99:   cbrApps.Add (onOffHelper.Install (nodes.Get (0)));
```

# Codes (6/7)

```
113:    // 8. Install FlowMonitor on all nodes
114:    FlowMonitorHelper flowmon;
115:    Ptr<FlowMonitor> monitor = flowmon.InstallAll();
116:
117:    // 9. Run simulation
118:    double SimTime = 10.0;
119:    Simulator::Stop (Seconds (SimTime));
120:    Simulator::Run ();
```

# Codes (7/7)

```
122:   // 10. Print per flow statistics
123:   monitor->CheckForLostPackets ();
124:   Ptr<Ipv4FlowClassifier> classifier = DynamicCast<Ipv4FlowClassifier>
     (flowmon.GetClassifier ());
125:   std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->GetFlowStats ();
126:   for (std::map<FlowId, FlowMonitor::FlowStats>::const_iterator i = stats.begin
     (); i != stats.end (); ++i)
127:     {
128:       // first 2 FlowIds are for ECHO apps, we don't want to display them
129:       if (i->first > 2)
130:         {
131:           Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow (i->first);
132:           std::cout << "Flow " << i->first - 2 << " (" << t.sourceAddress << " ->
     " << t.destinationAddress << ")\n";
133:           std::cout << "  Tx Bytes:   " << i->second.txBytes << "\n";
134:           std::cout << "  Rx Bytes:   " << i->second.rxBytes << "\n";
135:           std::cout << "  Throughput: " << i->second.rxBytes * 8.0 / SimTime /
     1000 / 1000  << " Mbps\n";
136:         }
137:     }
138:
139:   // 11. Cleanup
140:   Simulator::Destroy ();
141:   std::cout << "-----------------------------------------------\n";
142:   return 0;
143: }
```

# Result


root@localhost:~/桌面 /ns-allinone-3.10/ns-3.10 [157x57]
連線(C) 編輯(E) 檢視(V) 視窗(W) 選項(O) 說明(H)
[root@localhost ns-3.10]# ./waf --run scratch/p2-example
Waf: Entering directory `/root/桌面/ns-allinone-3.10/ns-3.10/build'
Waf: Leaving directory `/root/桌面/ns-allinone-3.10/ns-3.10/build'
'build' finished successfully (0.624s)
Flow 1 (192.168.0.1 -> 192.168.0.10)
Tx Bytes: 1155472
Rx Bytes: 1155472
Throughput: 0.924378 Mbps
[root@localhost ns-3.10]#

# Need to do…… (1/3)

- Transmission range of a node
  - ns3::RangePropagationLossModel
    - MaxRange
- Node moving speed
  - ns3::RandomWaypointMobilityModel
    - Speed
    - Pause
- Node data rate
  - ns3::ConstantRateWifiManager

# Need to do…… (2/3)

- RTS retry limit
  - ns3::UanMacRc
    - RetryRate
- Buffer size
  - ns3::TcpSocket
    - SndBufSize
- AODV hello periods
  - ns3::aodv::RoutingProtocol
    - HelloInterval

# Need to do…… (3/3)

- Uniform distribution in a square with 500 meters
  - ns3::RandomRectanglePositionAllocator
    - Attributes defined for this type:
      - **X**: A random variable which represents the x coordinate of a position in a random rectangle.
        - Set with class: **RandomVariableValue**
        - Underlying type: **RandomVariable**
        - Initial value: Uniform:0:1
        - Flags: construct write read
      - **Y**: A random variable which represents the y coordinate of a position in a random rectangle.
        - Set with class: **RandomVariableValue**
        - Underlying type: **RandomVariable**
        - Initial value: Uniform:0:1
        - Flags: construct write read
    - ns3::UniformVariable

# Q & A

